

PageRank with Restarts on WikiVote Network

Implementation and Conceptual Analysis

1 Introduction

In this project, we implemented PageRank with restarts on the WikiVote dataset. The goal was to analyze graph structure, implement both dense and sparse PageRank algorithms, handle dangling (deadend) nodes properly, and implement Personalized PageRank (PPR).

All implementations were done from scratch without using ready-made PageRank functions.

2 Graph Construction and Preprocessing

We worked with three graphs:

- Subgraph A (reindexed)
- Subgraph B (reindexed)
- Full WikiVote graph (original node IDs)

2.1 Edge Representation

For each graph, we construct adjacency matrix A such that:

$$A_{v,u} = 1 \quad \text{if edge } v \rightarrow u \text{ exists}$$

The transition matrix P is defined by row-normalization:

$$P_{v,u} = \begin{cases} \frac{1}{\text{outdeg}(v)} & \text{if } (v, u) \in E \\ 0 & \text{otherwise} \end{cases}$$

Dangling nodes (nodes with zero out-degree) produce zero rows.

2.2 Degree Analysis

For each graph we computed:

- Number of nodes (n)
- Number of edges (m)
- Number of self-loops
- In-degree and out-degree statistics
- Deadend count (out-degree = 0)

We plotted degree-rank distributions in log-log scale. The heavy-tailed behavior confirms that the network exhibits scale-free characteristics.

3 PageRank with Restarts

We use the standard PageRank iteration:

$$p_{t+1} = (1 - r)p_t P + r q$$

Where:

- r is the restart probability (0.15)
- q is the teleportation distribution
- p_t is the probability vector at iteration t

Convergence is measured using the L_1 residual:

$$\|p_{t+1} - p_t\|_1$$

The iteration stops when:

$$\|p_{t+1} - p_t\|_1 < 10^{-9}$$

4 Dense Implementation

For subgraphs A and B, we implemented a dense adjacency matrix representation.

4.1 Advantages

- Simple implementation
- Easy to debug

4.2 Disadvantages

- $O(n^2)$ memory
- Not scalable to large graphs

5 Sparse (CSR) Implementation

For efficiency, we implemented CSR (Compressed Sparse Row) representation.

Matrix multiplication was performed using:

$$p_{t+1} = (1 - r)P^T p_t + r q$$

CSR allows:

- $O(m)$ memory
- Fast sparse matrix-vector multiplication
- Scalability to large graphs

Timing measurements were collected for:

- CSR construction time
- Normalization time
- Iteration time

Sparse implementation is significantly faster and memory efficient compared to dense.

6 Deadend (Dangling) Nodes

Dangling nodes cause probability mass leakage:

$$\sum_i p_i < 1$$

We implemented two fixes:

6.1 Fix-q Method

Leaked probability mass is redistributed according to teleportation vector q :

$$p \leftarrow p + (1 - \sum_i p_i) q$$

6.2 Fix-u Method

Leaked probability mass is distributed uniformly:

$$p \leftarrow p + \frac{1 - \sum_i p_i}{n} \mathbf{1}$$

6.3 Observation

Both methods maintain $\sum_i p_i = 1$. However, they may produce different top-ranked nodes depending on network structure.

7 Personalized PageRank

We implemented PPR using seed node $s = 4037$.

Teleportation vector:

$$q_i = \begin{cases} 1 & \text{if } i = s \\ 0 & \text{otherwise} \end{cases}$$

This biases the random walk toward nodes close to the seed.

PPR reveals the local influence region of the selected node.

8 Implementation Details

8.1 Key Functions

- `make_row_stochastic_dense`
- `make_row_stochastic_csr`
- `pagerank_power_iteration`
- `run_sparse`

The algorithm supports both dense and sparse matrices using conditional checks.

8.2 Convergence Guarantee

With restart probability $r > 0$, the Markov chain becomes irreducible and aperiodic, ensuring convergence to a unique stationary distribution.

9 Complexity Analysis

9.1 Dense Version

Time per iteration = $O(n^2)$

Memory = $O(n^2)$

9.2 Sparse Version

Time per iteration = $O(m)$

Memory = $O(m)$

Since $m \ll n^2$ for real-world graphs, sparse is preferred.

10 Conclusion

We successfully implemented:

- Dense PageRank
- Sparse PageRank (CSR)
- Deadend handling (fix-q and fix-u)
- Personalized PageRank

The sparse implementation demonstrates significantly better scalability. Deadend handling is necessary to maintain valid probability distributions. Personalized PageRank highlights local structural importance in directed graphs.